

To-Do App Debugging Session

Key Concepts & Mistakes Summary
JavaScript / TypeScript • Event Handling • Modules • DOM

1. Key Concepts You Learned

1.1 ES Modules vs Global Scope

When a script is treated as an ES module (because of `export {}` or `type="module"`), functions declared inside it are **not** attached to the global window object. Inline HTML event handlers like `onclick="myFunction()"` look for functions only in the global scope. This is why you got:

```
Uncaught ReferenceError: addToCompleteTasksArray is not defined
```

Solutions: Either expose the functions with `window.fn = fn`, or (better) stop using inline onclick and use `addEventListener` + event delegation.

1.2 TypeScript DOM Typing

TypeScript is strict about DOM types:

- `querySelector / querySelectorAll` return `Element`
- `Element` does **not** have `.dataset` → needs cast to `HTMLElement`
- `e.target` is typed as `EventTarget`
- `EventTarget` does **not** have `.closest()` → needs cast to `Element`

```
const target = e.target as Element;
const btn = target.closest(".start-btn");
const index = Number((btn as HTMLButtonElement).dataset.index);
```

1.3 Event Delegation

Instead of attaching a listener to every button (which disappears when you re-render the HTML), you attach **one listener** on a parent (or document) and use `.closest()` to detect which button was clicked. You store the needed data in `data-*` attributes:

```
<button data-index="{i}" data-type="todo" data-action="progress" class="start-btn">
```

This pattern is essential when the DOM is recreated frequently (like after every task move).

2. Mistakes Made in the Code

Mistake #1 — Wrong data-action values

The click handler only understood three values: `"todo"`, `"complete"`, and `"progress"`. But some buttons were generated with `data-action="start"`, which matched nothing → clicking Start did nothing.

Column	Button	Wrong value	Correct value
To Do	Start	"start"	"progress"
To Do	Complete	"complete"	"complete" ✓
Progress	To Do	"todo"	"todo" ✓
Progress	Complete	"complete"	"complete" ✓
Complete	To Do	"complete" (wrong!)	"todo"
Complete	Start	"start"	"progress"

Mistake #2 — Adding the click listener on every re-render

You called `clickOnButtons()` inside `showToDoTasks()`, `showProgressTasks()`, and `showCompleteTasks()`. Every time a task moved, three new document-level listeners were added. This caused multiple executions and unpredictable behavior.

Correct approach: Call `clickOnButtons()` only **once** (right after the first `showAllTasks()`).

Mistake #3 — Using `=` instead of `+=` when building card HTML

In `showProgressTasks` and `showCompleteTasks` you wrote:

```
progressTasksCards = `...card HTML...` // overwrites every iteration
```

Only the last card survived. The To-Do column used `+=` correctly, which is why it showed all tasks.

Mistake #4 — Hardcoded content in the Complete column

Instead of using the actual task data, the Complete cards contained:

```
<h4>Yousti</h4>
<p>task description</p>
```

It should have been `${completeTasksArray[i].name}` and `${completeTasksArray[i].description}`.

3. Best Practices Going Forward

- **Prefer event delegation** — Attach one listener high in the DOM and use `data-*` attributes + `closest()`.
- **Keep listeners outside re-render functions** — Never re-attach document/window listeners every time you rebuild the UI.
- **Be consistent with data-action values** — Decide on a clear set of action names and use them everywhere.
- **Always use `+=` when building HTML strings in loops** — Assignment (`=`) keeps only the last item.
- **Cast carefully in TypeScript** — `Element` → `HTMLElement` for `dataset`; `EventTarget` → `Element` for `closest()`.
- **Avoid inline onclick with modules** — Use `addEventListener` so you stay inside the module scope.